EXPERIMENT 5

Academic Year: 2026-27 Year and Sem: TE V
Subject: DEV Lab Course Code: 2175119

PROBLEM STATEMENT:

To create, build, and manage Docker images and containers for application deployment.

THEORY:

Docker has revolutionized the world of software development by providing a lightweight and efficient way to package, distribute, and run applications in containers. This experiment aims to delve into Docker's architecture, comprehend the container life cycle, and gain hands-on experience by installing Docker and executing essential commands to manage images and interact with containers.

Key Concepts:

Docker Architecture:

- Docker Daemon:

The Docker daemon, running on the host machine, manages Docker containers. It listens for Docker API requests and takes care of building, running, and distributing Docker containers.

- Docker Client:

The Docker client is the primary interface through which users interact with Docker. It sends commands to the Docker daemon, initiating various Docker operations.

- Docker Images:

Docker images are lightweight, standalone, and executable packages that include everything needed to run a piece of software, including the code, runtime, libraries, and system tools.

- Docker Registry:

Docker images can be stored and shared through Docker registries. Docker Hub is a popular public registry, and private registries allow organizations to manage and distribute their images securely.

Container Life Cycle:

- Creating Containers:

Containers are instances of Docker images. They can be created from scratch or based on existing images. The process involves defining the container's characteristics and configuration.

- Running Containers:

Containers can be started with the docker run command, executing the processes defined in the image. Running containers are isolated from the host system, ensuring consistency across different environments.

- Pausing and Resuming Containers:

Containers can be paused to temporarily stop their execution without removing them. They can later be resumed from the paused state.

- Stopping and Removing Containers:

Containers can be stopped using docker stop and removed using docker rm. Stopping halts the container's execution, while removal deletes the container instance.

- Inspecting Containers:

The docker inspect command provides detailed information about a container, including its configuration, network settings, and volumes.

Docker Commands:

- Installing Docker:

Install Docker on the host machine by following the platform-specific installation instructions provided by Docker.

- Managing Images:

Use commands like docker pull to download images, docker build to create custom images, and docker push to upload images to a registry.

- Interacting with Containers:

Execute docker run to create and start a container, docker exec to run commands inside a running container, and docker ps to list running containers.

- Monitoring and Logs:

Monitor container resource usage with docker stats and view container logs using docker logs.

IMPLEMENTATION AND OUTPUT:

First, update your existing list of packages:
sudo apt update

Next, install a few prerequisite packages which let apt use packages over HTTPS:
sudo apt install apt-transport-https ca-certificates curl software-properties-common

Then add the GPG key for the official Docker repository to your system:
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo apt-key add -

Add the Docker repository to APT sources:
sudo add-apt-repository "deb [arch=amd64] https://download.docker.com/linux/ubuntu focal stable"

This will also update our package database with the Docker packages from the newly added repo.

Make sure you are about to install from the Docker repo instead of the default Ubuntu repo:

apt-cache policy docker-ce

You'll see output like this, although the version number for Docker may be different:

```
docker-ce:
  Installed: (none)
  Candidate: 5: 19.03.9~3-0~ubuntu-focal
  Version table:
     5: 19.03.9~3-0~ubuntu-focal 500
        500 https://download.docker.com/linux/ubuntu focal/stable amd64 Packages
```

Notice that docker-ce is not installed, but the candidate for installation is from the Docker repository for Ubuntu 20.04 (focal).

Finally, install Docker:

sudo apt install docker-ce

Docker should now be installed, the daemon started, and the process enabled to start on boot. Check that it's running:

sudo systemctl status docker

The output should be similar to the following, showing that the service is active and running:

```
Output
● docker.service - Docker Application Container Engine
     Loaded: loaded (/lib/systemd/system/docker.service; enabled; vendor preset: enabl
     Active: active (running) since Tue 2020-05-19 17:00:41 UTC; 17s ago
TriggeredBy: ● docker.socket
       Docs: https://docs.docker.com
   Main PID: 24321 (dockerd)
      Tasks: 8
     Memory: 46.4M
     CGroup: /system.slice/docker.service
             └─24321 /usr/bin/dockerd -H fd:// --containerd=/run/containerd/containerd
```

Pull an image from the Docker hub using the following command:

$ sudo docker run hello-world

```
sheetal@sheetal-ASUS-EXPERTBOOK-B1502CBA-B1502CBA:~$ sudo docker run hello-worldU
latest: Pulling from library/hello-world
c1ec31eb5944: Pull complete
Digest: sha256:4bd78111b6914a99dbc560e6a20eab57ff6655aea4a80c50b0c5491968cbc2e6
Status: Downloaded newer image for hello-world:latest

Hello from Docker!
This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:
 1. The Docker client contacted the Docker daemon.
 2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
    (amd64)
 3. The Docker daemon created a new container from that image which runs the
    executable that produces the output you are currently reading.
 4. The Docker daemon streamed that output to the Docker client, which sent it
    to your terminal.

To try something more ambitious, you can run an Ubuntu container with:
 $ docker run -it ubuntu bash

Share images, automate workflows, and more with a free Docker ID:
 https://hub.docker.com/

For more examples and ideas, visit:
 https://docs.docker.com/get-started/
```

Here, hello-world is the docker image present on the Docker hub.

8. Check if the docker image has been pulled and is present in your system using the following command:
$ sudo docker images

```
sheetal@sheetal-ASUS-EXPERTBOOK-B1502CBA-B1502CBA:~$ sudo docker images
REPOSITORY    TAG       IMAGE ID       CREATED        SIZE
hello-world   latest    d2c94e258dcb   9 months ago   13.3kB
sheetal@sheetal-ASUS-EXPERTBOOK-B1502CBA-B1502CBA:~$ sudo docker ps -a
CONTAINER ID   IMAGE         COMMAND    CREATED              STATUS                          PORTS     NAMES
ca79f472fd87   hello-world   "/hello"   About a minute ago   Exited (0) About a minute ago             crazy_mcnulty
sheetal@sheetal-ASUS-EXPERTBOOK-B1502CBA-B1502CBA:~$ sudo docker ps
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS     NAMES
sheetal@sheetal-ASUS-EXPERTBOOK-B1502CBA-B1502CBA:~$ sudo docker images
REPOSITORY    TAG       IMAGE ID       CREATED        SIZE
hello-world   latest    d2c94e258dcb   9 months ago   13.3kB
sheetal@sheetal-ASUS-EXPERTBOOK-B1502CBA-B1502CBA:~$ sudo docker images -a
REPOSITORY    TAG       IMAGE ID       CREATED        SIZE
hello-world   latest    d2c94e258dcb   9 months ago   13.3kB
sheetal@sheetal-ASUS-EXPERTBOOK-B1502CBA-B1502CBA:~$ sudo docker images --no-trunc
REPOSITORY    TAG       IMAGE ID                                                                  CREATED        SIZE
hello-world   latest    sha256:d2c94e258dcb3c5ac2798d32e1249e42ef01cba4841c2234249495f87264ac5a   9 months ago   13.3kB
```

To display all the containers pulled, use the following command:
$ sudo docker ps -a

To check for containers in a running state, use the following command:
$ sudo docker ps

You may use the -a switch to see a list of all Docker Photos, which comprises intermediary images.

sudo docker images -a

Typically, just the first twelve letters of an image's ID are shown when listing images for Docker. Utilize the -no-trunc switch to see images from Docker having extended Image Ids.

sudo docker images --no-trunc

The Docker Search Command: The typical syntax for a Docker search command is:

sudo docker search <image-name>

```
sheetal@sheetal-ASUS-EXPERTBOOK-B1502CBA-B1502CBA:~$ sudo docker search ubuntu
NAME                              DESCRIPTION                                     STARS     OFFICIAL
ubuntu                            Ubuntu is a Debian-based Linux operating sys…   16868     [OK]
websphere-liberty                 WebSphere Liberty multi-architecture images …   298       [OK]
open-liberty                      Open Liberty multi-architecture images based…   64        [OK]
neurodebian                       NeuroDebian provides neuroscience research s…   106       [OK]
ubuntu-debootstrap                DEPRECATED; use "ubuntu" instead                52        [OK]
ubuntu-upstart                    DEPRECATED, as is Upstart (find other proces…   115       [OK]
ubuntu/nginx                      Nginx, a high-performance reverse proxy & we…   112
ubuntu/squid                      Squid is a caching proxy for the Web. Long-t…   83
ubuntu/cortex                     Cortex provides storage for Prometheus. Long…   4
ubuntu/prometheus                 Prometheus is a systems and service monitori…   56
ubuntu/apache2                    Apache, a secure & extensible open-source HT…   70
ubuntu/kafka                      Apache Kafka, a distributed event streaming …   39
ubuntu/bind9                      BIND 9 is a very flexible, full-featured DNS…   74
ubuntu/mysql                      MySQL open source fast, stable, multi-thread…   59
ubuntu/zookeeper                  ZooKeeper maintains configuration informatio…   13
ubuntu/postgres                   PostgreSQL is an open source object-relation…   35
ubuntu/jre                        Distroless Java runtime based on Ubuntu. Lon…   13
ubuntu/redis                      Redis, an open source key-value store. Long-…   22
ubuntu/dotnet-aspnet              Chiselled Ubuntu runtime image for ASP.NET a…   17
ubuntu/grafana                    Grafana, a feature rich metrics dashboard & …   9
ubuntu/dotnet-deps                Chiselled Ubuntu for self-contained .NET & A…   13
ubuntu/memcached                  Memcached, in-memory keyvalue store for smal…   5
ubuntu/dotnet-runtime             Chiselled Ubuntu runtime image for .NET apps…   14
ubuntu/prometheus-alertmanager    Alertmanager handles client alerts from Prom…   9
ubuntu/cassandra                  Cassandra, an open source NoSQL distributed …   2
```

Execute the following command to download the official ubuntu image to your computer:

Sudo docker pull ubuntu

```
sheetal@sheetal-ASUS-EXPERTBOOK-B1502CBA-B1502CBA:~$ sudo docker pull ubuntu
[sudo] password for sheetal:
Using default tag: latest
latest: Pulling from library/ubuntu
57c139bbda7e: Pull complete
Digest: sha256:e9569c25505f33ff72e88b2990887c9dcf230f23259da296eb814fc2b41af999
Status: Downloaded newer image for ubuntu:latest
docker.io/library/ubuntu:latest
sheetal@sheetal-ASUS-EXPERTBOOK-B1502CBA-B1502CBA:~$
```

As an example, let's run a container using the latest image of Ubuntu. The combination of the **-i** and **-t** switches gives you interactive shell access into the container:

docker run -it ubuntu

```
sheetal@sheetal-ASUS-EXPERTBOOK-B1502CBA-B1502CBA:~$ sudo docker pull ubuntu
[sudo] password for sheetal:
Using default tag: latest
latest: Pulling from library/ubuntu
57c139bbda7e: Pull complete
Digest: sha256:e9569c25505f33ff72e88b2990887c9dcf230f23259da296eb814fc2b41af999
Status: Downloaded newer image for ubuntu:latest
docker.io/library/ubuntu:latest
sheetal@sheetal-ASUS-EXPERTBOOK-B1502CBA-B1502CBA:~$ sudo docker images
REPOSITORY    TAG       IMAGE ID       CREATED        SIZE
ubuntu        latest    fd1d8f58e8ae   2 weeks ago    77.9MB
hello-world   latest    d2c94e258dcb   9 months ago   13.3kB
sheetal@sheetal-ASUS-EXPERTBOOK-B1502CBA-B1502CBA:~$ sudo docker run -it ubuntu
root@4a740a1cc566:/#
```

Now you can run any command inside the container. For example, let's update the package database inside the container. You don't need to prefix any command with sudo, because you're operating inside the container as the **root** user:

apt update

```
sheetal@sheetal-ASUS-EXPERTBOOK-B1502CBA-B1502CBA:~$ sudo docker run -it ubuntu
root@4a740a1cc566:/# apt update
Get:1 http://archive.ubuntu.com/ubuntu jammy InRelease [270 kB]
Get:2 http://security.ubuntu.com/ubuntu jammy-security InRelease [110 kB]
Get:3 http://archive.ubuntu.com/ubuntu jammy-updates InRelease [119 kB]
Get:4 http://archive.ubuntu.com/ubuntu jammy-backports InRelease [109 kB]
Get:5 http://archive.ubuntu.com/ubuntu jammy/multiverse amd64 Packages [266 kB]
Get:6 http://archive.ubuntu.com/ubuntu jammy/restricted amd64 Packages [164 kB]
Get:7 http://archive.ubuntu.com/ubuntu jammy/universe amd64 Packages [17.5 MB]
Get:8 http://security.ubuntu.com/ubuntu jammy-security/restricted amd64 Packages [1779 kB]
Get:9 http://security.ubuntu.com/ubuntu jammy-security/universe amd64 Packages [1065 kB]
Get:10 http://security.ubuntu.com/ubuntu jammy-security/multiverse amd64 Packages [44.6 kB]
Get:11 http://security.ubuntu.com/ubuntu jammy-security/main amd64 Packages [1453 kB]
Get:12 http://archive.ubuntu.com/ubuntu jammy/main amd64 Packages [1792 kB]
Get:13 http://archive.ubuntu.com/ubuntu jammy-updates/restricted amd64 Packages [1822 kB]
Get:14 http://archive.ubuntu.com/ubuntu jammy-updates/multiverse amd64 Packages [50.4 kB]
Get:15 http://archive.ubuntu.com/ubuntu jammy-updates/universe amd64 Packages [1339 kB]
Get:16 http://archive.ubuntu.com/ubuntu jammy-updates/main amd64 Packages [1735 kB]
Get:17 http://archive.ubuntu.com/ubuntu jammy-backports/universe amd64 Packages [28.1 kB]
Get:18 http://archive.ubuntu.com/ubuntu jammy-backports/main amd64 Packages [50.4 kB]
Fetched 29.7 MB in 7s (4010 kB/s)
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
2 packages can be upgraded. Run 'apt list --upgradable' to see them.
root@4a740a1cc566:/#
```

Then install any application in it. Let's install Node.js:

apt install nodejs

```
root@4a740a1cc566:/# apt install nodejs
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following additional packages will be installed:
  ca-certificates javascript-common libbrotli1 libc-ares2 libicu70 libjs-highlight.js libnghttp2-14 libnode72 libuv1 nodejs-doc openssl
Suggested packages:
  apache2 | lighttpd | httpd npm
The following NEW packages will be installed:
  ca-certificates javascript-common libbrotli1 libc-ares2 libicu70 libjs-highlight.js libnghttp2-14 libnode72 libuv1 nodejs nodejs-doc openssl
0 upgraded, 12 newly installed, 0 to remove and 2 not upgraded.
Need to get 26.1 MB of archives.
After this operation, 93.0 MB of additional disk space will be used.
Do you want to continue? [Y/n] y
Get:1 http://archive.ubuntu.com/ubuntu jammy-updates/main amd64 openssl amd64 3.0.2-0ubuntu1.14 [1184 kB]
Get:2 http://archive.ubuntu.com/ubuntu jammy-updates/main amd64 ca-certificates all 20230311ubuntu0.22.04.1 [155 kB]
Get:3 http://archive.ubuntu.com/ubuntu jammy/main amd64 libicu70 amd64 70.1-2 [10.6 MB]
Get:4 http://archive.ubuntu.com/ubuntu jammy-updates/main amd64 libnghttp2-14 amd64 1.43.0-1ubuntu0.1 [76.7 kB]
Get:5 http://archive.ubuntu.com/ubuntu jammy/main amd64 libuv1 amd64 1.43.0-1 [93.1 kB]
Get:6 http://archive.ubuntu.com/ubuntu jammy/main amd64 javascript-common all 11+nmu1 [5936 B]
Get:7 http://archive.ubuntu.com/ubuntu jammy/main amd64 libbrotli1 amd64 1.0.9-2build6 [315 kB]
Get:8 http://archive.ubuntu.com/ubuntu jammy/universe amd64 libjs-highlight.js all 9.18.5+dfsg1-1 [367 kB]
Get:9 http://archive.ubuntu.com/ubuntu jammy-updates/main amd64 libc-ares2 amd64 1.18.1-1ubuntu0.22.04.2 [45.0 kB]
Get:10 http://archive.ubuntu.com/ubuntu jammy-updates/universe amd64 libnode72 amd64 12.22.9~dfsg-1ubuntu3.3 [10.8 MB]
Get:11 http://archive.ubuntu.com/ubuntu jammy-updates/universe amd64 nodejs-doc all 12.22.9~dfsg-1ubuntu3.3 [2410 kB]
Get:12 http://archive.ubuntu.com/ubuntu jammy-updates/universe amd64 nodejs amd64 12.22.9~dfsg-1ubuntu3.3 [122 kB]
Fetched 26.1 MB in 7s (3633 kB/s)
debconf: delaying package configuration, since apt-utils is not installed
Selecting previously unselected package openssl.
(Reading database ... 4393 files and directories currently installed.)
Preparing to unpack .../00-openssl_3.0.2-0ubuntu1.14_amd64.deb ...
Unpacking openssl (3.0.2-0ubuntu1.14) ...
Selecting previously unselected package ca-certificates.
Preparing to unpack .../01-ca-certificates_20230311ubuntu0.22.04.1_all.deb ...
Unpacking ca-certificates (20230311ubuntu0.22.04.1) ...
Selecting previously unselected package libicu70:amd64.
Preparing to unpack .../02-libicu70_70.1-2_amd64.deb ...
Unpacking libicu70:amd64 (70.1-2) ...
Selecting previously unselected package libnghttp2-14:amd64.
Preparing to unpack .../03-libnghttp2-14_1.43.0-1ubuntu0.1_amd64.deb ...
Unpacking libnghttp2-14:amd64 (1.43.0-1ubuntu0.1) ...
Selecting previously unselected package libuv1:amd64.
Preparing to unpack .../04-libuv1_1.43.0-1_amd64.deb ...
Unpacking libuv1:amd64 (1.43.0-1) ...
Selecting previously unselected package javascript-common.
Preparing to unpack .../05-javascript-common_11+nmu1_all.deb ...
Unpacking javascript-common (11+nmu1) ...
Selecting previously unselected package libbrotli1:amd64.
Preparing to unpack .../06-libbrotli1_1.0.9-2build6_amd64.deb ...
Unpacking libbrotli1:amd64 (1.0.9-2build6) ...
Selecting previously unselected package libjs-highlight.js.
```

This installs Node.js in the container from the official Ubuntu repository. When the installation finishes, verify that Node.js is installed:

node -v

```
root@4a740a1cc566:/# node -v
v12.22.9
root@4a740a1cc566:/#
```

Any changes you make inside the container only apply to that container.

To exit the container, type exit at the prompt.

```
root@4a740a1cc566:/# exit
exit
sheetal@sheetal-ASUS-EXPERTBOOK-B1502CBA-B1502CBA:~$
```

After using Docker for a while, you'll have many active (running) and inactive containers on your computer. To view the **active ones**, use:

docker ps

```
sheetal@sheetal-ASUS-EXPERTBOOK-B1502CBA-B1502CBA:~$ sudo docker ps
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS     NAMES
sheetal@sheetal-ASUS-EXPERTBOOK-B1502CBA-B1502CBA:~$
```

To view all containers — active and inactive, run docker ps with the -a switch:

docker ps -a

```
sheetal@sheetal-ASUS-EXPERTBOOK-B1502CBA-B1502CBA:~$ sudo docker ps -a
CONTAINER ID   IMAGE         COMMAND       CREATED          STATUS                          PORTS     NAMES
4a740a1cc566   ubuntu        "/bin/bash"   7 minutes ago    Exited (0) About a minute ago             distracted_lumiere
ca79f472fd87   hello-world   "/hello"      28 minutes ago   Exited (0) 28 minutes ago                 crazy_mcnulty
sheetal@sheetal-ASUS-EXPERTBOOK-B1502CBA-B1502CBA:~$
```

To view the latest container you created, pass it the -l switch:

docker ps -l

```
sheetal@sheetal-ASUS-EXPERTBOOK-B1502CBA-B1502CBA:~$ sudo docker ps -l
CONTAINER ID   IMAGE     COMMAND       CREATED          STATUS                       PORTS     NAMES
4a740a1cc566   ubuntu    "/bin/bash"   8 minutes ago    Exited (0) 2 minutes ago             distracted_lumiere
sheetal@sheetal-ASUS-EXPERTBOOK-B1502CBA-B1502CBA:~$
```

To start a stopped container, use docker start, followed by the container ID or the container's name. Let's start the Ubuntu-based container with the ID of 1c08a7a0d0e4:

docker start 1c08a7a0d0e4

```
heetal@sheetal-ASUS-EXPERTBOOK-B1502CBA-B1502CBA:~$ sudo docker start 4a740a1cc566
a740a1cc566
heetal@sheetal-ASUS-EXPERTBOOK-B1502CBA-B1502CBA:~$ sudo docker ps
ONTAINER ID   IMAGE     COMMAND       CREATED          STATUS          PORTS     NAMES
a740a1cc566   ubuntu    "/bin/bash"   10 minutes ago   Up 32 seconds             distracted_lumiere
heetal@sheetal-ASUS-EXPERTBOOK-B1502CBA-B1502CBA:~$
```

When you start up a Docker image, you can create, modify, and delete files just like you can with a virtual machine. The changes that you make will only apply to that container. You can start and stop it, but once you destroy it with the docker rm command, the changes will be lost for good.

This section shows you how to save the state of a container as a new Docker image.

After installing Node.js inside the Ubuntu container, you now have a container running off an image, but the container is different from the image you used to create it. But you might want to reuse this Node.js container as the basis for new images later.

Then commit the changes to a new Docker image instance using the following command.

docker commit -m "What you did to the image" -a "Author Name" container_id repository/new_image_name

The **-m** switch is for the commit message that helps you and others know what changes you made, while **-a** is used to specify the author. The container_id is the one you noted earlier in the tutorial when you started the interactive Docker session. Unless you created additional repositories on Docker Hub, the repository is usually your Docker Hub username.

For example, for the user **sammy**, with the container ID of d9b100f2f636, the command would be:

docker commit -m "added Node.js" -a "sammy" d9b100f2f636 sammy/ubuntu-nodejs

When you *commit* an image, the new image is saved locally on your computer. Later in this tutorial, you'll learn how to push an image to a Docker registry like Docker Hub so others can access it.

Listing the Docker images again will show the new image, as well as the old one that it was derived from:

docker images

```
sheetal@sheetal-ASUS-EXPERTBOOK-B1502CBA-B1502CBA:~$ sudo docker commit -m "added Node.js" -a "sheetal" 4a740a1cc566 sheetal/ubuntu-nodejs
[sudo] password for sheetal:
sha256:ae7a5d521eb224f633c6ad3d6134fdce8022ca6d9806872129ca49740bdfe58b
sheetal@sheetal-ASUS-EXPERTBOOK-B1502CBA-B1502CBA:~$ sudo docker images
REPOSITORY              TAG       IMAGE ID       CREATED          SIZE
sheetal/ubuntu-nodejs   latest    ae7a5d521eb2   58 seconds ago   210MB
ubuntu                  latest    fd1d8f58e8ae   2 weeks ago      77.9MB
hello-world             latest    d2c94e258dcb   9 months ago     13.3kB
sheetal@sheetal-ASUS-EXPERTBOOK-B1502CBA-B1502CBA:~$ sudo docker login -u sheetalj18
Password:
WARNING! Your password will be stored unencrypted in /root/.docker/config.json.
Configure a credential helper to remove this warning. See
https://docs.docker.com/engine/reference/commandline/login/#credentials-store

Login Succeeded
sheetal@sheetal-ASUS-EXPERTBOOK-B1502CBA-B1502CBA:~$ sudo docker push sheetalj18/ubuntu-nodejs
Using default tag: latest
The push refers to repository [docker.io/sheetalj18/ubuntu-nodejs]
An image does not exist locally with the tag: sheetalj18/ubuntu-nodejs
sheetal@sheetal-ASUS-EXPERTBOOK-B1502CBA-B1502CBA:~$ docker tag sheetal/ubuntu-nodejs sheetalj18/ubuntu-nodejs
permission denied while trying to connect to the Docker daemon socket at unix:///var/run/docker.sock: Post "http://%2Fvar%2Frun%2Fdocker.soc
u-nodejs&tag=latest": dial unix /var/run/docker.sock: connect: permission denied
sheetal@sheetal-ASUS-EXPERTBOOK-B1502CBA-B1502CBA:~$ sudo docker tag sheetal/ubuntu-nodejs sheetalj18/ubuntu-nodejs
sheetal@sheetal-ASUS-EXPERTBOOK-B1502CBA-B1502CBA:~$ sudo docker push sheetalj18/ubuntu-nodejs
Using default tag: latest
The push refers to repository [docker.io/sheetalj18/ubuntu-nodejs]
bb0f6da4cec4: Pushed
1a102d1cac2b: Mounted from library/ubuntu
latest: digest: sha256:29a9e92c913a95b6b0bd79f2a6dd461b315b940222385df2f0ad1103895e011f size: 741
sheetal@sheetal-ASUS-EXPERTBOOK-B1502CBA-B1502CBA:~$
```

To push your image, first log into Docker Hub.

docker login -u docker-registry-username

You'll be prompted to authenticate using your Docker Hub password. If you specified the correct password, authentication should succeed.

Note: If your Docker registry username is different from the local username you used to create the image, you will have to tag your image with your registry username. For the example given in the last step, you would type:

docker tag sammy/ubuntu-nodejs docker-registry-username/ubuntu-nodejs

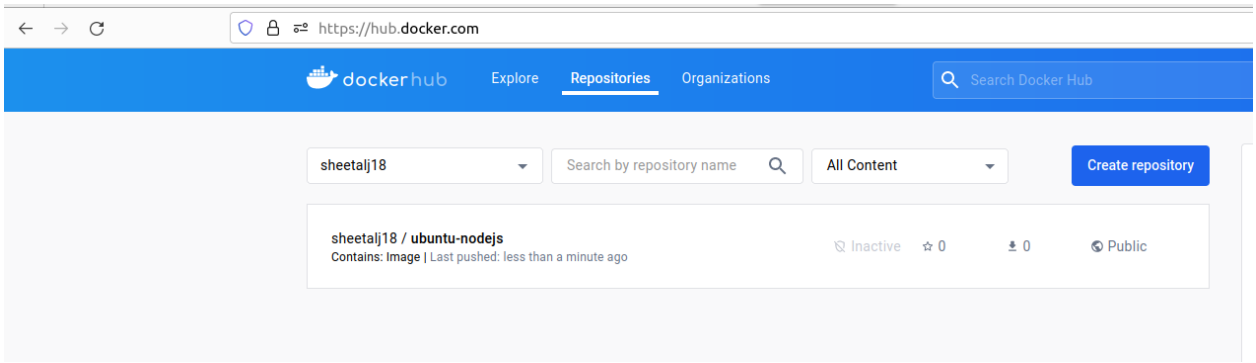
Then you may push your own image using:

docker push docker-registry-username/docker-image-name

To push the ubuntu-nodejs image to the **sammy** repository, the command would be:

docker push sammy/ubuntu-nodejs

The process may take some time to complete as it uploads the images,



CONCLUSION:

Understanding Docker's architecture and the container life cycle is fundamental for harnessing the full potential of containerization in software development. By installing Docker and executing key commands, this experiment empowers participants to manage Docker images, create containers, and interact with them seamlessly. The knowledge gained in this experiment lays a solid foundation for incorporating Docker into the software development workflow, promoting consistency and scalability across diverse computing environments.